ADAM (A Decentralized Approach Model) İşbirlikçi Konsensüsü

1. Giriş ve Arka Plan

ADAM (A Decentralized Approach Model) konsensüs mekanizması; blok zincirini madencilik merkezileşmesine, bencil madenciliğe (selfish mining), blok grinding işlemlerine ve blok liderlerine yönelik hedefli Denial-of-Service (DoS) saldırılarına karşı güçlendirmek için tasarlanmış işbirlikçi bir hibrit konsensüs modelidir.

Geleneksel Proof-of-Work (PoW) ve Proof-of-Stake (PoS) protokolleri belirgin güvenlik açıklarına sahiptir:

- PoW modelinde, devasa işlem (hashing) gücüne sahip madenciler blok üretimine hakim olur ve bu durum konsensüsü merkezileştiren havuzların oluşmasına yol açar.
- PoS modelinde, en zengin stake eden cüzdanların blok üretme olasılığı orantısız şekilde daha yüksektir.
- Her iki modelde de, bir sonraki blok üreticisinin kimliği önceden biliniyorsa (önceden seçilmişse), DoS saldırılarının veya rüşvet girişimlerinin hedefi haline gelirler. Eğer blok içerikleri bir sonraki bloğun rastgeleliğini değiştirecek şekilde manipüle edilebiliyorsa, madenciler gelecekteki lider seçimlerini saptırmak için *grinding saldırıları* gerçekleştirebilirler.

ADAM, blok üretim sürecini bir **İşbirlikçi Hibrit Turuna** bölerek bu sorunları çözer:

1. **Seçim Aşaması (Election Phase)**: Her blok slotu için, sözde rastgele bir seed deterministik olarak N **adet Madenci (Miners)** ve **1 adet Koordinatör (Coordinator)** havuzunu seçer.
2. **Çözme Aşaması (Solving Phase)**: Seçilen Madenciler, mevcut bloğun hedefine bağlı olarak hafif, paralel hale getirilmiş PoW bulmacalarını (kısmi çözümler) derler ve çözer.
3. **Birleştirme Aşaması (Aggregation Phase)**: Seçilen Koordinatör kısmi çözümleri toplar, doğrular, bunları bir blok şablonu (block template) halinde paketler, kendi gizli anahtarını (private key) kullanarak nihai blok başlığını (block header) imzalar ve bunu ağa yayınlar.

---

2. Konsensüs Mimarisi ve Parametreleri

ADAM konsensüsü, blok yüksekliğine bağlı olarak koşullu bir şekilde etkinleştirilir. Temel parametreler `src/consensus/params.h` dosyasında tanımlanmıştır ve ağ bazında `src/chainparams.cpp` dosyasında yapılandırılmıştır.

Aktif Masternode sayısının düşük olduğu durumlarda ağın sorunsuz bir şekilde bootstrap (ilk kurulum) yapabilmesini sağlamak için ADAM, bir ağ Spork'u tarafından dinamik olarak kontrol edilen iki çalışma modunu destekler.

A. Fallback Mode (Geri Çekilme Modu - Blok Versiyon 11)

Fallback Mode, ağın bootstrap aşaması için tasarlanmıştır. Aktif Masternode kaydı gerektirmeden çalışır ve tamamen kendi kendine yeten (self-contained) bir yapıdadır.

- **Madenci Sayısı (N)**: $11 \le K \le 14$ olmak üzere, $K-1$ boyutundaki anahtar havuzuna (key pool) göre dinamik olarak belirlenir (anahtarlar deterministiktir ve seçilen Koordinatör `vAdamMiners` dizisinin son elemanı olarak eklenir).
- **Konsensüs Eşiği (T)**: Her iki modda da `nAdamThreshold` konsensüs parametresinde sabitlenmiştir (bu değer Mainnet/Regtest üzerinde `7` iken, yerel testleri/blok koordinasyonunu kolaylaştırmak amacıyla Testnet üzerinde `3` olarak ayarlanmıştır).
- **Kendi Kendine Yeten Başlık Düzeni**: `vAdamMiners`, tüm K açık anahtarlarını depolar (ilk $K-1$ seçilen madenciler, son anahtar koordinatördür). `vAdamSolutions` ise $K-1$ kısmi çözümü depolar.

B. Standart Mod (Blok Versiyon 12)

Standart Mod, tam bir Masternode ağının mevcut olmasını gerektiren, eksiksiz işbirlikçi konsensüs durumunu temsil eder.

- **Madenci Sayısı (N)**: `nAdamMinersCount` konsensüs parametresi aracılığıyla yapılandırılır (`src/consensus/params.h` dosyasında tanımlanmış ve `src/chainparams.cpp` içinde Mainnet/Testnet/Regtest için `11` olarak başlatılmıştır).
- **Konsensüs Eşiği (T)**: `nAdamThreshold` konsensüs parametresi aracılığıyla yapılandırılır (`src/consensus/params.h` dosyasında tanımlanmış ve `src/chainparams.cpp` içinde Mainnet/Regtest için `7`, Testnet için `3` olarak başlatılmıştır).
- **Seçilen Koordinatör**: Koordinatör, aktif Masternode listesinden dinamik olarak seçilir ve madenci listesinden farklıdır.

C. Spork Kontrollü Etkinleştirme (`SPORK_21_ADAM_STANDARD_MODE`)

Fallback Mode (Versiyon 11) ile Standart Mod (Versiyon 12) arasındaki geçiş `SPORK_21_ADAM_STANDARD_MODE` (Spork ID `10020`) tarafından kontrol edilir.

- **Varsayılan Değer**: `4070908800ULL` (KAPALI).
- **Davranış**:
  - Spork aktif ise: Bloklar, Standart Mod kuralları altında **Versiyon 12** olarak oluşturulur.
  - Spork aktif değilse: Bloklar, Fallback Mode kuralları altında **Versiyon 11** olarak oluşturulur.

Ağ Yapılandırmaları ve Aktivasyon Yükseklikleri

İş birlikçi konsensüs çerçevesinin geçişi üç ana aktivasyon eşiğinden oluşur:

1. `Consensus::UPGRADE_ADAM`: ADAM Fallback Modunu (Blok Sürümü 11) etkinleştirir.
2. `Consensus::UPGRADE_POMBL`: ADAM Standart Modunu / Çoklu Kanıt Algoritmasını (Multi-Proof-Algorithm - Blok Sürümü 12) etkinleştirir.
3. `Consensus::UPGRADE_MODELD`: Model D ödül dağılımlarını ve PoBLS konsensüsünü zorunlu kılar.

Ağ	`UPGRADE_ADAM` (ADAM ve Çift Kulvar Başlangıcı)	`UPGRADE_ADAM_V2` (Shift V2)	`UPGRADE_POMBL` (Standart Mod / V12)	`UPGRADE_MODELD` (Model D / PoBLS)	Hedef Blok Süresi
Mainnet	Blok 200 (`nAdamDualTrackHeight = 200`)	Blok 705	Blok 2000	Blok 2200	30 saniye
Testnet	Blok 200 (`nAdamDualTrackHeight = 200`)	Blok 300	Blok 400	Blok 500	30 saniye
Regtest	Blok 200 (`nAdamDualTrackHeight = 1`)	Blok 705	Blok 300	Blok 200	10 saniye

D. Bulmaca Zorluğu Bit Kaydırma (Bit-Shift) Parametreleri

Farklı blok yüksekliklerinde dinamik ve yapılandırılabilir zorluk ölçeklendirmesini korurken, ağ üzerinde bulmaca çözümlerinin "yanlış pozitif seline" (false-positive flood) yol açmasını önlemek için ADAM, `Consensus::Params` tarafından yönetilen `GetAdamDifficultyShift(nHeight, consensus)` (`src/adam.cpp`) fonksiyonunu kullanır:

- **V1 Zorluk Kaydırması (`nAdamDifficultyShiftV1`)**: `Consensus::UPGRADE_ADAM_V2` öncesindeki blok yükseklikleri için gevşetilmiş hedef zorluğun bit kaydırma çarpanı. Değer: `10`.
- **V2 Zorluk Kaydırması (`nAdamDifficultyShiftV2`)**: `Consensus::UPGRADE_ADAM_V2` ve üzerindeki blok yükseklikleri için gevşetilmiş hedef zorluğun bit kaydırma çarpanı. Değer: `6`.
- **Yükseltme Aktivasyonu (`Consensus::UPGRADE_ADAM_V2`)**: Zorluk kaydırma çarpanının `V1` (`10`) değerinden `V2` (`6`) değerine geçtiği ağ yükseltme yüksekliği (Mainnet/Regtest'te `705`, Testnet'te `300`).

Bulmaca doğrulaması sırasında, `scaledTarget` değeri, konsensüs hedefinin (`Target`) mevcut aktif zorluk kaydırma değeri kadar kaydırılmasıyla elde edilir:

scaledTarget = Target ⋘ activeShift

- `NetworkUpgradeActive(nHeight, consensus, Consensus::UPGRADE_ADAM_V2)` `false` ise activeShift=nAdamDifficultyShiftV1 (`10`).
- `NetworkUpgradeActive(nHeight, consensus, Consensus::UPGRADE_ADAM_V2)` `true` ise activeShift=nAdamDifficultyShiftV2 (`6`).

E. Başlangıç Zorluk Limiti (powLimit)

1–199 arasındaki blokların çok hızlı kazılmasını (bu durum çatallanmalara ve kuorum kilitlenmelerine yol açıyordu) önlemek için, başlangıç zorluk hedefi `powLimit` şu şekilde ayarlanmıştır:

powLimit = UINT256_ZERO ⋙ 20

Mainnet ve Testnet üzerinde bu değer tam olarak `1/2^20`'dir (genesis bloğunun `0x1e0ffff0` olan `nBits` değerine eşdeğerdir). Bu, blokların genesis'ten itibaren doğal olarak yaklaşık 30 saniye aralıklarla yerleşmesini sağlayarak düğümlerin (nodes) kararlı P2P bağlantıları kurmasına ve birleşik bir zincir ucunu (chain tip) sürdürmesine olanak tanır.

---

3. Doğrulanabilir Rastgele Fonksiyon (VRF) ve Rolling Seed Yapısı

Madencilerin blok H hash'ini manipüle etmek için işlemleri veya nonce'ları değiştirmesini ve böylece blok $H + 1$ için yapılacak lider seçimini saptırmasını (yani **grinding saldırılarını**) önlemek amacıyla ADAM, bir **Doğrulanabilir Rastgele Fonksiyon (VRF) Rolling Seed** modeli uygular.

Matematiksel Formülasyon

ADAM ağ yükseltmesinin (`Consensus::UPGRADE_ADAM`) aktif olduğu herhangi bir H blok yüksekliği için:

$$\text{Seed}_H = \text{Hash}(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1})$$

Burada:

- H yüksekliğindeki seçim seed'i (Seed), blok $H + 1$ için liderleri belirlemek üzere kullanılır.
- $H - 1$ yüksekliğindeki VRF kanıtı (VRFProof), $H - 1$ bloğunun seçilen **Koordinatörü** tarafından, hibrit bir BLS12-381 + ECDSA fallback imza mekanizması (`SignBLSWithECDSAFallback`) kullanılarak $H - 2$ yüksekliğindeki önceki seed üzerinde oluşturulan imzadır.
- Bu şema altında Koordinatör, kendi ECDSA gizli anahtarından (private key) deterministik olarak türetilen bir BLS gizli anahtarını kullanarak bir BLS12-381 imzası oluşturur. Yetkilendirmeyi kanıtlamak ve anahtar manipülasyonunu önlemek için Koordinatör, BLS açık anahtarını (public key) kendi ECDSA gizli anahtarını kullanarak imzalar. Doğrulama (`VerifyBLSWithECDSAFallback`) işlemi, hem BLS anahtarının ECDSA yetkilendirmesinin hem de hedef seed'in BLS imzasının geçerli olmasını sağlar.
- Elde edilen imza deterministik olduğundan, Koordinatör'ün belirli bir seed için tam olarak bir geçerli imzası vardır; bu da rolling seed yapısını, Koordinatör imzalayıp yayınlayana kadar tamamen değiştirilemez ve tahmin edilemez kılar.

Düğüm Seçimi (SSLE)

Madencilerin ve koordinatörün seçimi `src/adam.cpp` içindeki `SelectAdamNodes()` tarafından gerçekleştirilir:

1. Aktif düğüm havuzu (kayıtlı Masternode listesi ve Coin-Lock veya PoW-Lock aracılığıyla aktif kayıtlı madenciler) derlenir.

2. Seçim havuzu ağa bağlıdır:

    - **Mainnet & Testnet**: Havuz, aktif Masternode'lardan ve aktif kayıtlı madencilerden dinamik olarak oluşturulur. Bununla birlikte, aktif masternode'lar veya kayıtlar tam olarak kurulmadan önce blok üretiminin kararlı kalmasını sağlamak amacıyla, blok yüksekliği bootstrap limitinin altında olduğu sürece (Mainnet üzerinde `nAdamBootstrapLimit` = 44.850, Testnet üzerinde 100.000 blok), ağ, blok 1 ila 199 arasındaki blok üreticilerini (coinbase çıktıları) otomatik olarak tarar ve açık anahtarlarını madenci havuzuna ekler. Bu, erken aşamalarda zincirin durmasını (stall) önler.
    - **Regtest**: Havuz, otomatik testleri kolaylaştırmak amacıyla harici kayıtları otomatik olarak atlar ve 15 adet deterministik bootstrap açık anahtarı içerir:

      $$\text{Pool}_{\text{bootstrap}} = \{\text{DeterministicPubKey}_0, \ldots, \text{DeterministicPubKey}_{14}\}$$

3. **Rolling seed'e dayanarak seçim havuzundaki her bir düğüm için benzersiz bir hash sırası (hash rank) hesaplanır**:

$\text{Rank}_i = \text{Hash}(\text{Seed}_H \| \text{PubKey}_i)$

4. **Havuz,** Rank_i **değerlerine göre artan düzende sıralanır**.

5. **Dual-Track (**8 + 3**) Komite Seçimi (**`nAdamDualTrackHeight` **= Mainnet/Testnet üzerinde 200, Regtest üzerinde 1; mevcut bir zincirde etkinleştirilirken daima mevcut yüksekliğin ilerisinde bir blok yüksekliğine** ≥UPGRADE_ADAM **ayarlanmalıdır)**: `IsAdamDualTrackActive(H)` aktif olduğunda (`H >= consensus.nAdamDualTrackHeight`), 11 doğrulayıcı koltuğu çift yönlü dinamik taşma (spillover) mekanizmasına sahip `ElectDualTrackValidators()` aracılığıyla iki kulvara ayrılır:

   - **Kulvar 1 — Teminatlı Kulvar (** `ADAM_COLLATERALIZED_TRACK_QUOTA = 8` **koltuk)**:
     - **Alt-Kulvar 1A (** `ADAM_MASTERNODE_TRACK_QUOTA = 7` **koltuk)**: **Aktif Masternode'lara (** `4.200 KRISTA` **teminatlı)** ayrılmıştır. Derleme anında `static_assert(ADAM_MASTERNODE_TRACK_QUOTA >= 7)` ile kilitlenmiştir; böylece Aktif Masternode'lar bootstrap sonrasında da (`GetAdamThreshold = 7`) tek başına kuorum oluşturabilir.
     - **Alt-Kulvar 1B (** `ADAM_COIN_LOCK_TRACK_QUOTA = 1` **koltuk)**: `250 KRISTA` **Coin-Lock** doğrulayıcılarına ayrılmıştır.
     - Alt-Kulvar 1A ile Alt-Kulvar 1B arasındaki doğrudan kulvar-içi taşma (spillover) sayesinde, havuzda hiç Coin-Lock olmadığında Aktif Masternode'lar Kulvar 1'in 8 koltuğunun tamamını (`7 + 1 = 8`) doldurur; Aktif Masternode sayısı `< 7` olduğunda ise boş koltuklar Açık PoW kulvarına taşmadan önce Coin-Lock doğrulayıcıları tarafından doldurulur.
   - **Kulvar 2 — Açık PoW Kulvarı (** `ADAM_OPEN_POW_TRACK_QUOTA = 3` **koltuk)**: Açık PoW madencilerine (`10 KRISTA` CLTV kilitli `PoW-Lock`, `Zero-Coin PoW` ve `Blok 1..199` bootstrap anahtarları) ayrılmıştır. Havuzda 3'ten az Açık PoW adayı varsa, Kulvar 2'deki boş koltuklar önce kalan Aktif Masternode'lara, ardından kalan Coin-Lock doğrulayıcılarına devredilir.
   - Bu yapı, 11 doğrulayıcı koltuğunun her koşulda eksiksiz doldurulmasını garanti ederken `4.200 KRISTA` Aktif Masternode'ları korur (`7` doğrulayıcı koltuğu + `1` Koordinatör), `250 KRISTA` Coin-Lock doğrulayıcılarına `10 KRISTA` `PoW-Lock` seyreltmesinden izole `1` koltuklu özel bir teminatlı alt-kulvar sunar ve Açık PoW madencilerini `3` koltuk (`%27,27`) ile sınırlar.
   - **Yedek Doğrulayıcı Koltuk Devri ve Zamana Bağlı Kademeli Eşik Gevşetme (** `ResolveAdamCommitteeForTime` **,** `ValidateAdamCommitteeForTime` **,** `GetAdamEffectiveThreshold` **)**:
     - Bir bloğun üretimi ≥ 60 sn gecikirse (`ADAM_COMMITTEE_FALLBACK_TIMEOUT_SECS = 60`, her `ADAM_COMMITTEE_FALLBACK_SLOT_SECS = 30` saniyelik slotta genişleyerek), 11 kişilik komitede çözüm göndermeyen sessiz koltuklar, bloğun kendi `CCoinsViewCache* pView` görünümüne göre yalnızca **aynı kulvardaki** (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Açık PoW -> Açık PoW`) ve o kulvar içindeki sıkı sıralı yedek doğrulayıcılara (`k_t > prevStandbyIdxInTrack[t]` ve `k_t < nReplacedInTrack[t] + nSlot`) devredilebilir.
     - Model D (`IsModelDActive`) kapsamında **Eşit Emeğe Eşit Kazanç** prensibi uygulanır: 11 kişilik komitedeki her $i \in \{0, \ldots, 10\}$ koltuğu, `vAdamSolutions[i]` içinde geçerli bir PoW çözümü sunduğu sürece — ister asıl birincil doğrulayıcı `vExpectedMiners[i]` tarafından ister asıl koltuk zaman aşımına uğradığında ($t \geq 60$ sn) devreye giren aynı kulvardaki meşru yedek doğrulayıcı `vAdamMiners[i]` tarafından çözülmüş olsun — `%40`'lık Katılımcı Doğrulayıcı ödül havuzundan kendi kulvar payını (7 Masternode onaylayıcı koltuğu için toplam `%20` [koltuk başı `~%2,857`] + 4 bağımsız madenci koltuğu için toplam `%20` [1 Coin-Lock `%8,00` + 3 Açık PoW koltuk başı `%4,00`]) alır. Çözümsüz/boş kalan koltuklar (`vAdamSolutions[i].empty()`) `%0` alır; yedek doğrulayıcıyla değiştirilen her koltuk

(`vAdamMiners[i] != vExpectedMiners[i]`) için ise Koordinatörün `%15` payından ilgili koltuğun ödülüne eşit tutarda kesinti (`nCoordSubstitutionPenalty`) yapılır (`0` net koordinatör kazancı). Hem boş koltuk payları hem de koordinatör ikame kesintisi (`nUnclaimedPartSplit = (nPartSplitTotal - nPaidPartTotal) + nCoordSubstitutionPenalty`) **Geliştirici Hazinesine (`nExpectedTreasury = nTreasurySplit + nUnclaimedPartSplit`)** aktarılırken Pasif Masternode Kazananı tam `%35` (`nMNSplit`) payını alır. Ayrıca LLMQ blok önerisi doğrulaması (`qblockprop`, yalnızca düğümün mevcut ucu `msg.block.hashPrevBlock` ile eşleştiğinde değerlendirilir) sırasında $t \geq 60$ sn olduğunda, önerilen blokta bir birincil koltuk i yedek anahtarla **değiştirilmişse** (`vAdamMiners[i] != vExpectedMiners[i]`) ve LLMQ üyesinde asıl doğrulayıcı `vExpectedMiners[i]`'ye ait geçerli bir çözüm mevcutsa, LLMQ üyesi öneriyi reddeder ve `QUORUMSIGSHARE` imzalamaz (boş koltuklar `vAdamMiners[i] == vExpectedMiners[i]` P2P yarış koşullarından dolayı dürüst hızlı blokları geciktirmemek adına veto edilmez).

- Blok üretimi ≥ 120 sn (`ADAM_THRESHOLD_RELAXATION_TIMEOUT_SECS = 120`) boyunca tıkanırsa, `GetAdamEffectiveThreshold(nHeight, nTimeElapsed, nActiveMasternodes)` gereken kuorum eşiğini her `60 saniyede` (`ADAM_THRESHOLD_RELAXATION_SLOT_SECS = 60`) `1` azaltarak bootstrap sonrasında (`nHeight >= nAdamThresholdBootstrapLimit`) VEYA havuzdaki aktif Masternode sayısı ≥ 7 olduğunda asgari `ADAM_MIN_RELAXED_THRESHOLD = 5` tabanına (`> 1 Coin-Lock + 3 Açık PoW = 4`, `static_assert` ve çalışma anı `assert` ile kilitli; her zaman en az 1 Aktif Masternode zorunlu), yalnızca bootstrap döneminde aktif Masternode sayısı < 7 iken `ADAM_MIN_RELAXED_THRESHOLD_BOOTSTRAP = 3` tabanına kadar gevşetir. Yalnızca başlık doğrulaması yapan `ContextualCheckBlockHeader` (blok UTXO görünümü henüz mevcut olmadığından senkronizasyon/reorg sırasında yanlış banlamayı önlemek için) `nActiveMasternodes` için `0` geçerken, tam blok doğrulaması (`CheckFormsOfWork`) bloğun kendi `pView` görünümüne göre `GetActiveMasternodePoolCount(nHeight - 1, pView)` değerini kesin olarak uygular.
- Düğüm genelinde sürümden bağımsız ebeveyn-blok koruyucusu (`src/miner.cpp` içindeki `ActiveProposalGuard`), aynı düğümün aynı `(parentHash, coordinatorKey)` için hem PoW hem PoS bloğunu eşzamanlı veya ardışık üretmesini tüm blok sürümlerinde (`v11` ve `v12`) engeller.

6. **Koordinatör Seçimi**:

- **Kural 1 (`Aktif Masternode Sayısı > Eşik`)**: En yüksek sıralamaya sahip Aktif Masternode **Koordinatör** olarak seçilir; kalan havuzdan 11 **Madenci** `ElectDualTrackValidators` ile seçilir.
- **Kural 2 (`Aktif Masternode Sayısı <= Eşik`)**: 11 **Madenci** `ElectDualTrackValidators` ile seçilir ve havuzda seçilmeyen en yüksek sıralamalı anahtar **Koordinatör** olarak atanır.

---

4. Blok Başlığı Uzantıları ve Serileştirme (Serialization)

ADAM ağ yükseltmesi (`Consensus::UPGRADE_ADAM`) aktif olduğunda, bloklar **Versiyon 11** (Fallback Mode) veya **Versiyon 12** (Standart Mod) blok yapıları kullanılarak serileştirilir. `src/primitives/block.h` dosyasındaki `CBlockHeader` sınıfı dört yeni alanla genişletilmiştir:

```
class CBlockHeader {
public:
    // Legacy fields...
    int32_t nVersion;
    uint256 hashPrevBlock;
    uint256 hashMerkleRoot;
    uint32_t nTime;
    uint32_t nBits;
    uint32_t nNonce;

    // ADAM Extended fields (Version >= 11)
    std::vector<CPubKey> vAdamMiners;                // Public keys of elected miners (and
        coordinator in v11)
    std::vector<std::vector<unsigned char>> vAdamSolutions; // Serialized partial solutions
        (nonce + miner signature)
    std::vector<unsigned char> vAdamVRFProof;        // Coordinator's VRF signature on
        previous seed
    std::vector<unsigned char> vAdamCoordinatorSig;  // Coordinator's signature on final
        block header hash
};
```

4.1 Hashing ve Serileştirme Sıralaması

Nihai blok başlığı hash'ini imzalayan ve `GetHash()` fonksiyonunun dışında tutulan `vAdamCoordinatorSig` alanının aksine, `vAdamVRFProof` blok hash'i hesaplanmadan *önce* üretilir. `GetHash()` sırasında blok başlığı serileştirmesine dahil edilir, bu da VRF kanıtının başlığa kriptografik olarak kilitlenmesini sağlar.

4.2 Durumsuz Blok Hashing Zinciri (Stateless)

Blok hash'ini (`CBlockHeader::GetHash()`) hesaplamak için ADAM, serileştirilmiş başlığı (versiyon 12 ise `vAdamCoordinatorSig` ve LLMQ `vQuorumSig` hariç tutularak), seçilen madencilere karşılık gelen M adet ardışık hashing turundan oluşan bir zincir boyunca hash'ler:

- Fallback modunda (`nVersion == 11`): M = vAdamMiners.size() − 1.
- Standart modda (`nVersion == 12`): M = vAdamMiners.size().

Her bir $i \in \{0, \ldots, M-1\}$ turu için:

1. Önceki blok hash'inden ve tur endeksinden benzersiz bir tur hash'i türetilir:

   $\text{roundHash}_i = \text{Hash}(\text{hashPrevBlock} \| i)$

2. İlk bayt $v_i = \text{roundHash}_i[0]$ elde edilir.

3. Tek bir ortak asal (coprime) çarpanı m_i hesaplanır:

   $m_i = v_i \mid 1$

   Eğer $m_i < 3$ ise, $m_i = 3$ olarak ayarlanır. Bu, çarpanların 2^{256} değerine göre aralarında asal olmasını sağlayarak %100 entropiyi korur.

4. Tur hashing işlemi gerçekleştirilir:

   - **Fallback Modu (Versiyon 11)**:
     - **Tur 0**: $H_0 = \text{CalculateAdamPuzzleHash}(\text{algo}_0, \text{SerializedHeader}) \times m_0 \pmod{2^{256}}$

- **Tur $i > 0$:** $H_i = \text{CalculateAdamPuzzleHash}(\text{algo}_i, H_{i-1}) \times m_i \pmod{2^{256}}$

- **Standart Mod (Versiyon 12)**:
  - **Tur 0**:
    - Madenci 0 için 3-permütasyon algoritmaları algo1, algo2, algo3 türetilir.
    - Hesaplama yapılır:

      $H_0^{(3)} = \text{CalculateAdamPuzzleHash}(\text{algo3}, \text{SerializedHeader})$

      $H_0^{(2)} = \text{CalculateAdamPuzzleHash}(\text{algo2}, (H_0^{(3)} \times 1) \pmod{2^{256}})$

      $H_0 = \text{CalculateAdamPuzzleHash}(\text{algo1}, (H_0^{(2)} \times 1) \pmod{2^{256}})$
    - Çarpan uygulanır:

      $H_{\text{prev}} = (H_0 \times m_0) \pmod{2^{256}}$
  - **Tur $i > 0$**:
    - Madenci i için 3-permütasyon algoritmaları algo1, algo2, algo3 türetilir.
    - Hesaplama yapılır:

      $H_i^{(3)} = \text{CalculateAdamPuzzleHash}(\text{algo3}, H_{\text{prev}})$

      $H_i^{(2)} = \text{CalculateAdamPuzzleHash}(\text{algo2}, (H_i^{(3)} \times (i+1)) \pmod{2^{256}})$

      $H_i = \text{CalculateAdamPuzzleHash}(\text{algo1}, (H_i^{(2)} \times (i+1)) \pmod{2^{256}})$
    - Çarpan uygulanır:

      $H_{\text{prev}} = (H_i \times m_i) \pmod{2^{256}}$

Elde edilen nihai hash H_{prev} (veya Versiyon 11'de $H_0 \times m_0$), blok hash'i olarak döndürülür.

---

5. Konsensüs Doğrulama Kuralları

Bir blok alındığında, ADAM ağ yükseltmesi (`Consensus::UPGRADE_ADAM`) aktifse, `src/main.cpp` içindeki `CheckBlock()` fonksiyonu aşağıdaki doğrulamaları zorunlu kılar:

1. **Versiyon Zorunluluğu ve Sürüm Düşürme (Downgrade) Koruması**:
   - Blok veriyonu en az `11` olmalıdır.
   - Eğer `block.nVersion == 11` and `block.GetBlockTime() >= sporkManager.GetSporkValue(SPORK_21_ADAM_STANDARD_MODE)` ise, kötü niyetli madencilerin sürüm düşürmesini önlemek amacıyla blok bir `bad-version` DoS hatası ile reddedilir.
   - Blok yüksekliği ≥nPoMBLHeight ise (Regtest üzerinde 300, Testnet üzerinde 400, Mainnet üzerinde 2000), Standart Mod aktif olduğu takdirde blok veriyonu `12` olmalıdır.
2. **Madenciler ve Çözüm Boyutları**:

- **Versiyon 11 (Fallback Mode)**:
  - `vAdamMiners` boyutu 11 ile 14 arasında olmalıdır.
  - `vAdamSolutions` boyutu tam olarak `vAdamMiners.size() - 1` olmalıdır.
  - Doğrulanacak Koordinatör, `vAdamMiners` dizisinin son elemanıdır (`vAdamMiners.back()`).
- **Versiyon 12 (Standart Mod)**:
  - `vAdamMiners` boyutu tam olarak `nAdamMinersCount` değerine eşit olmalıdır.
  - `vAdamSolutions` boyutu tam olarak `vAdamMiners.size()` değerine eşit olmalıdır.
  - Koordinatör, `SelectAdamNodes` algoritmasından türetilir.

3. **VRF Kanıtı Doğrulaması**: `vAdamVRFProof`, önceki rolling seed Seed_{H-1}'e göre doğrulanmalıdır ve `VerifyBLSWithECDSAFallback` kullanılarak beklenen Koordinatörün açık anahtarıyla eşleşen geçerli bir imza olmalıdır.

4. **Seçim Yolu Doğrulaması (Election Path Validation)**: Versiyon 12'de, `vAdamMiners` içindeki açık anahtar listesi deterministik `SelectAdamNodes` algoritmasının tam çıktısıyla eşleşmelidir. Versiyon 11, kendi kendine yeten bootstrap modunda çalıştığı için bu kontrolü atlar.

5. **Kısmi Çözümler Doğrulaması (Partial Solutions Validation)**:
   - Geçerli kısmi çözümlerin sayısı `GetAdamThreshold(nHeight)` fonksiyonunun zorunlu kıldığı eşiği karşılamalıdır:
     - **Versiyon 11 & Versiyon 12**: Bootstrap aşamasında (Mainnet'te blok yüksekliği < 50.000, Testnet'te < 100.000) kesintisiz ağ canlılığını garantiye almak için 5 geçerli çözüm, 50.000. bloktan itibaren ise tam `nAdamThreshold` konsensüs parametresi uyarınca 7 çözüm (Testnet üzerinde 3 çözüm).
   - Her çözüm bir `nonce` ve bir `signature` (imza) olarak ayrıştırılır.
   - Bulmaca hash'i (puzzle hash), madenciye atanan algoritma(lar) kullanılarak hesaplanır:
     - **Versiyon 11 (Fallback Mode)** içinde:

       $\text{PuzzleHash} = \text{CalculateAdamPuzzleHash}(\text{algoIndex}, \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i)$

       Burada $\text{algoIndex} = \text{Hash}(\text{Seed}_H \| \text{MinerPubKey}_i) \pmod{18}$ olup, desteklenen **18 algoritmadan** biri kullanılır (bunlar arasında `Hamsi`, `Fugue`, `Shabal`, `Whirlpool` ve `Haval-256` yer alır).

     - **Versiyon 12 (Standart Mod)** içinde: Önceki bloğun hash'ine ve madencinin açık anahtarına dayanarak mevcut 18 algoritma arasından deterministik olarak 3 farklı hashing algoritmasını (algo1, algo2 ve algo3) seçen bir 3-permütasyon seçici şeması `\text{GetAdam3PermutationAlgos}(\text{hashPrevBlock}, \text{MinerPubKey}_i)` kullanılır. Çözücü bu üç algoritmayı birleştirir:

       $\text{PuzzleHash} = \text{algo1}((\text{minerIdx} + 1) \times \text{algo2}((\text{minerIdx} + 1) \times \text{algo3}(\text{Challenge}))) \pmod{2^{256}}$

       Burada $\text{Challenge} = \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i$ şeklindedir.

     - `PuzzleHash`, `nBits` ile tanımlanan hedef zorluğu karşılamalıdır (Bölüm 2.D'de açıklandığı gibi `scaledTarget = Target \ll \text{activeShift}` olarak gevşetilir).
   - İmza, `PuzzleHash`'i imzalayan `MinerPubKey_i`'ye göre doğrulanmalıdır.

6. **Koordinatör İmzası Doğrulaması**: `vAdamCoordinatorSig`, `VerifyBLSWithECDSAFallback` kullanılarak nihai blok başlığı hash'ini (imzanın kendisi hariç) imzalayan beklenen Koordinatörün açık anahtarına göre doğrulanmalıdır.

---

6. Cüzdan ve RPC API Entegrasyonu

Geliştiricileri ve madencilik havuzu operatörlerini desteklemek amacıyla RPC API, işbirlikçi konsensüs alanlarını dışarıya sunar (expose):

`getblocktemplate` Yanıtı

`nVersion >= 11` olduğunda, JSON yanıtı şunları içerir:

- `version`: 11
- `adamminers`: Seçilen madencilerin hex-kodlu açık anahtarlarından oluşan dizi.
- `adamvrfproof`: Önceki bloğun seed'i üzerindeki hex-kodlu VRF imzası.
- `adamsolutions`: Şu anda toplanan hex-kodlu kısmi çözümlerden oluşan dizi.
- `adamcoordinatorsig`: Koordinatörün hex-kodlu imzası.

`generate` RPC Güncellemesi

`generate` RPC'si, ADAM blokları için blok birleştirme (block assembly) işlemini gerçekleştirir. Geleneksel yüksek zorluklu hash grinding işlemi yapmak yerine:

1. Rolling seed'e dayanarak düğümün bir Madenci mi yoksa Koordinatör mü olarak seçildiğini belirler.
2. Madenci olarak seçildiyse, hafif kısmi PoW bulmacasını çözer.
3. Koordinatör olarak seçildiyse çözümleri toplar, VRF kanıtını ve nihai blok hash'ini deterministik olarak imzalar ve tamamlanmış blok şablonunu ağa gönderir.

---

7. Proof-of-Stake (PoS) Entegrasyonu ve İşbirlikçi PoS

ADAM, Proof-of-Stake (PoS) mekanizmasının yerine geçmez, aksine onunla entegre olarak PoS ile PoW'un (ADAM işbirlikçi madenciliği) birlikte çalıştığı bir **Hibrit İşbirlikçi PoS (Hybrid Cooperative PoS)** konsensüs mekanizması oluşturur.

Geleneksel PoS modelinde, blok üretimi yalnızca stake ağırlığına (cüzdanda tutulan coin miktarı) göre belirlenir. ADAM'da bu süreç, blok grinding'i, bencil staking'i ve hedefli lider DoS saldırılarını önlemek için işbirlikçi madenci-koordinatör doğrulama döngüsüyle birleştirilir.

Staking ve İşbirlikçi Yaşam Döngüsü (Blok yüksekliği $\geq$ 200)

Ağ yükseltmesi `Consensus::UPGRADE_POS` etkinleştirildiğinde (Mainnet üzerinde 200. blok yüksekliğinde), blok üretimi saf İşbirlikçi PoW'dan, PoW işbirlikçi madenciliği ile PoS staking'in paralel olarak çalıştığı Hibrit İşbirlikçi PoS modeline geçer:

> **ÖNEMLİ**
>
> **Tek İmzalı PoS Bloklarına İzin Verilmez:** Proof-of-Work (ADAM işbirlikçi madenciliği) asla sonlanmaz veya PoS lehine devre dışı bırakılmaz. KristaTech bünyesindeki her PoS bloğu hibrittir; seçilen ADAM madencileri tarafından çözülen PoW bulmacalarının doğrulanmasını gerektirir ve hem stake edenin anahtarı (`vchBlockSig`) hem de seçilen ADAM koordinatörünün anahtarı (`vAdamCoordinatorSig`) ile çift imzalanmış (double-signed) olması gerekir. Tek imzalı bloklar, konsensüs doğrulama kuralları tarafından kesin bir şekilde reddedilir.

1. **Staking Yetkisi (Kernel Kontrolü)**: Cüzdanın staking iş parçacığı (staking thread - `ThreadStakeMinter`), kernel hash kontrolünü (coin ağırlığı ile orantılı olarak) doğrulayarak herhangi bir UTXO'nun bir blokta stake etmeye uygun olup olmadığını periyodik olarak değerlendirir.

2. **İşbirlikçi Bulmaca Toplama**: Bir staking iş parçacığı blok önerme hakkını kazandığında, bir blok şablonu (block template) oluşturur (blok yüksekliği ≥ 2000 olduğunda hem PoS hem de `UPGRADE_POMBL` aktif olduğu için Versiyon 12 olarak). Stake edenin cüzdanı, `SelectAdamNodes` aracılığıyla mevcut blok yüksekliği için seçilen madencileri alır ve bu seçilen madenciler tarafından çözülen hafif PoW bulmacalarını P2P ağı bellek önbelleğinden (`mapAdamSolutionsCache`) toplar. Seçilen herhangi bir madencinin çözümü yerel önbellekte eksikse, gerekli tüm çözümler alınana kadar blok şablonu ertelenir.

3. **Koordinatör Doğrulaması ve İmza**: Mevcut turun seçilen Koordinatörü blok şablonunu doğrular, VRF kanıtını (`vAdamVRFProof`) oluşturmak üzere önceki bloğin seed'ini imzalar ve Koordinatör imzasını (`vAdamCoordinatorSig`) oluşturmak üzere blok başlığını imzalar.

4. **Stake Eden Blok İmzası (Çift Kilitleme)**: Tamamlanan blok, iki ayrı imza türü kullanılarak güvence altına alınır:

   - **ADAM İmzası (`vAdamCoordinatorSig`)**: İşbirlikçi PoW madencilik turunun başarıyla tamamlandığını doğrulamak için seçilen Koordinatör tarafından oluşturulur.
   - **PoS Blok İmzası (`vchBlockSig`)**: Stake edenin cüzdanı tarafından `SignBlock` kullanılarak oluşturulur (stake edilen UTXO'nun gizli anahtarı kullanılarak nihai blok hash'i imzalanır).

Ağ Üzerinde Çift Doğrulama (Dual Validation)

Bir eş düğüm (peer) İşbirlikçi bir PoS bloğu aldığında, `CheckBlock` içindeki doğrulama kuralları her iki konsensüs kontrolünün de geçmesini zorunlu kılar:

1. **Proof-of-Stake Doğrulaması**: Düğüm, `coinstake` işlemini doğrular, kernel hash hedef zorluğunu kontrol eder ve stake edenin blok imzasını (`vchBlockSig`) doğrular.
2. **ADAM İşbirlikçi PoW Doğrulaması**: Düğüm, seçilen madencilerin kısmi PoW bulmaca çözümlerini (T kuorum eşiğini karşılayan `vAdamSolutions`), Koordinatörün VRF kanıtını (`vAdamVRFProof`), Koordinatörün imzasını (`vAdamCoordinatorSig`) ve Model D aktif olduğunda Versiyon 12 bloklarındaki LLMQ kuorum imzasını (`vQuorumSig`) doğrular.

---

8. LLMQ (Long-Living Masternode Quorums) ve DKG (Distributed Key Generation)

Standart Mod altında Versiyon 12 blok şablonlarını doğrulamak ve imzalamak için ağ, **Dağıtık Anahtar Üretimi (DKG - Distributed Key Generation)** oturumlarını yürüten **Uzun Ömürlü Masternode Kuorumlarına (LLMQ - Long-Living Masternode Quorums)** dayanır.

> **NOT**
> **Eşik Değerleri Arasındaki Fark: ADAM İşbirlikçi Madencilik Eşiği** (madenci bulmaca çözümleri `vAdamSolutions` için uygulanan 11'de 7 kuralı) ile **LLMQ İmza Doğrulama Eşiği** (blok imzası `vQuorumSig` için uygulanan 5'te 3 kuralı) karıştırılmamalıdır:
>
> - **ADAM Eşiği**: 11 seçilmiş madenciden en az 7'sinin (`nAdamThreshold = 7` Mainnet/Regtest üzerinde, `3` Testnet üzerinde; `nAdamThresholdBootstrapLimit` altındaki başlangıç aşamasında ise `5`) PoW bulmaca çözümünü (`vAdamSolutions`) çözmüş ve göndermiş olmasını şart koşar. Bu kural ADAM işbirlikçi bulmaca doğrulama katmanına aittir.
> - **LLMQ Eşiği**: Blok hash'inin geçerli sayılabilmesi için 5 LLMQ üyesinden en az 3'ünün (`quorum.members.size() / 2 + 1`) secp256k1 gizli anahtarlarıyla bloğu imzalamış olmasını (`vQuorumSig`) şart koşar. Bu kural merkeziyetsiz blok doğrulama katmanına aittir.

A. Kuorum Topolojisi ve Parametreleri

- **Kuorum Boyutu**: Her aktif LLMQ tam olarak **5 üyeden** oluşur (`llmq.cpp:218`).
- **DKG Aralıkları ve Blok Bazlı Rotasyon**:
  - **Başlangıç Epoch Modu**: Mainnet'te her **100 blokta** bir ve Testnet/Regtest'te her **10 blokta** bir yeni bir DKG oturumu gerçekleştirilir (`GetActiveQuorum`).
  - **Blok Bazlı Kuorum Rotasyonu**: Kuorum rotasyonu aktifleştiğinde (Mainnet'te **1.400**, Testnet'te **1.000** ve Regtest'te **800** blok yüksekliğinde), `GetActiveQuorum` her blok yüksekliğinde `((nHeight / dkgInterval) * dkgInterval) - 1` yüksekliğindeki epoch tohumunu kullanarak `RunDKG(nHeight)` çalıştırır.
- **Kuorum İmzası Doğrulama Eşiği (`CQuorumSignature::Verify`)**:
  - **Tüm Ağlar (Mainnet, Testnet, Regtest)**: Model D (`UPGRADE_MODELD`) aktif olduğunda, `CQuorumSignature::Verify` fonksiyonu BFT çoğunluk eşiğini (`quorum.members.size() / 2 + 1` , en az 2 ve en fazla kuorum boyutu olacak şekilde) zorunlu kılar; bu da 5 üyeli standart bir kuorum için en az **5'te 3** imza gerektirir.
  - **Model D Aktivasyonundan Önce**: Mainnet (2.000–2.199 blokları) ve Testnet (400–499 blokları) üzerinde başlangıç masternode birikimine olanak tanımak amacıyla kuorum imzası doğrulaması `CheckBlock()` içinde (`fFallbackMode`) atlanır. Regtest üzerinde Model D yüksekliği 200'de (`UPGRADE_POMBL` yüksekliği 300'den önce) aktifleştiği için 5'te 3 çoğunluk eşiği 300. bloktan itibaren her zaman aktiftir.

B. Aktif Masternode Filtreleme (%100 Zincir Üstü Deterministik Havuz)

- Kuorumlar ve ADAM doğrulayıcı seçimleri, yerel P2P bellek listesine (`mnodeman`) bağımlı olmaksızın %100 zincir üstü (on-chain) doğrulanmış Masternode durumundan (`mapMasternodeCollateralOutpoints`, `mapOnChainCollateralDelegations` ve `pcoinsTip` üzerindeki 4.200 KRISTA teminat doğrulaması) deterministik olarak seçilir.
- 5'ten az zincir üstü aktif masternode kayıtlıysa ağ, kuorum üyelerini deterministik olarak kayıtlı madenci havuzundan (`GetAdamMinerPool(nHeight - 1)` — blok 1-199 bootstrap madenciliği veya coin-lock/pow-lock kayıtları) seçmeye geri döner.
- Mainnet, Testnet veya Regtest üzerinde kod içinde sabitlenmiş (hardcoded) herhangi bir anahtar kimliği (key ID) filtrelemesi veya yerel cüzdan kısıtlaması uygulanmaz, bu da tamamen merkeziyetsiz ve güven gerektirmeyen (trustless) bir konsensüs ortamı sağlar.

C. Kayan Pencere Kuorum Rotasyonu (Sliding Window Quorum Rotation)

Kuorum üyelerini hedefli Hizmet Dışı Bırakma (DoS) ve DDoS saldırılarına karşı korumak için **Kayan Pencere Kuorum Rotasyonu** mekanizması uygulanmıştır. Şema altında:

- Her dönem sabit bir statik kuorum kullanmak veya tamamen yeni bir üye seti seçmek yerine, kuorum üyeleri blok yüksekliğine dayalı kayan bir kaydırma (sliding offset) kullanılarak masternode havuzundan seçilir:

  $\text{member_index}_i = (\text{nHeight} + i) \pmod{\text{total_masternodes}}$

- Her yeni blok yüksekliğinde (`nHeight`), seçim offseti 1 birim kayar; bu da önceki bloktaki **5 üyeden 4'ünün korunması** (konsensüs sürekliliği ve DKG el sıkışma durumunun korunması için) ve tam olarak **1 üyenin rotasyona uğraması** anlamına gelir.

- Bu, hedefli saldırıları ekonomik olarak anlamsız ve son derece karmaşık hale getirir, çünkü aktif üyeler saldırgan onları tespit edip ağ trafiğiyle boğmaya fırsat bulamadan sürekli değişir.

Rotasyon yükseltmesi ağ bazında belirlenen hedef blok yüksekliklerinde aktif hale gelir:

- **Regtest**: Blok yüksekliği ≥ 800
- **Testnet**: Blok yüksekliği ≥ 1000
- **Mainnet**: Blok yüksekliği ≥ 1400

---

9. Çevrimdışı Koordinatör Geri Çekilme Rotasyonu (Offline Coordinator Fallback Rotation)

Bootstrap aşamasında (Fallback Modu, Blok Versiyonu 11, blok yüksekliği < 2000 için aktif), birincil seçilen koordinatörün çevrimdışı olması durumunda ağın kilitlenmesini önlemek için tasarlanmıştır:

A. Geri Çekilme Aktivasyon Gecikmesi (Fallback Activation Delay)

Önceki bloğun süresinden bu yana geçen süre **45 saniyeyi** (`ADAM_FALLBACK_COORDINATOR_TIMEOUT_SECS = 45`) aşarsa, stake edenler ve madenciler birincil seçilen koordinatörün çevrimdışı veya ulaşılamaz olduğunu belirler.

B. Rotasyon Protokolü (Rotation Protocol)

Aktivasyon tetiklendiğinde, koordinatörlük görevi her **30 saniyede** bir mevcut blok slotundaki **11 seçilmiş madenci** (`vExpectedMiners` , sıra 0 ila 10) arasında döndürülür:

$$\text{rotationIndex} = \left(\frac{\text{timeElapsed} - 45}{30}\right) \pmod{11}$$

Ağ üzerindeki `vExpectedMiners[rotationIndex]` açık anahtarına karşılık gelen gizli anahtara sahip olan staker veya madenci, dinamik olarak blok şablonu oluşturma, VRF imzalama ve blok başlığı yayınlama görevini devralır.

C. Konsensüs Uyumluluğu (Consensus Compatibility)

Versiyon 11 blok doğrulama kuralları (`src/main.cpp` içindeki `CheckBlock` fonksiyonu), koordinatörün illa birincil seçilen koordinatör olmasını şart koşmadığı için (sadece blokta belirtilen koordinatörün imzası ve VRF geçerliliği kontrol edilir), bu rotasyon mekanizması ağın çevrimdışı liderleri atlayarak konsensüs bölünmesi veya sert çatal (hard fork) riski olmadan ilerlemesini sağlar.

D. Standart Mod Sıkı Doğrulaması (Blok Versiyonu 12)

Standart Modda (Blok Versiyonu 12, blok yüksekliği ≥ 2000 için aktif), normal şartlar altında on-chain üzerinde sıkı deterministik koordinatör seçimi zorunlu kılınmıştır. Koordinatör, `SelectAdamNodes` tarafından seçilen beklenen koordinatörle tam olarak eşleşmelidir.

Ancak, ağdaki aktif ve etkinleştirilmiş masternode sayısının kritik bir eşiğin altına (spesifik olarak < 11) düşmesi durumunda ağın donmasını önlemek amacıyla, Model D yükseltmesinin aktif olduğu blok yüksekliklerinde (≥ 2200 Mainnet, ≥ 500 Testnet veya ≥ 200 Regtest, `IsModelDActive` ile kontrol edilir) ağ dinamik olarak **Geri Çekilme Modunu (Fallback Mode)** aktif hale getirir (bu durum **Masternode Fallback Mode / Yöntem B #2** olarak adlandırılır).

Masternode Fallback Modu kapsamında:

1. Blok doğrulama kuralları **Fallback doğrulama kurallarına** geçer (geri çekilme koordinatör rotasyonuna izin verilir ve M = vAdamMiners.size() − 1 adet çözümün varlığı kontrol edilir).
2. Blok şablonu, koordinatör anahtarı `vAdamMiners` dizisinin sonuna eklenerek oluşturulur.
3. Seçilen koordinatör çevrimdışıysa, fallback koordinatör rotasyon protokolü (Bölüm 9.B) Versiyon 12 blokları için de devreye girerek koordinatörlük görevini seçilen madenciler arasında dinamik olarak döndürür.

Bu çift modlu konsensüs kuralı, ciddi masternode kesintilerinde dahi ağın kendi kendini iyileştirmesini ve yüksek kullanılabilirlikte kalmasını sağlar.

---

10. Konsensüs Kesinliği, Çatal Çözümü (Fork Resolution) ve Performans İyileştirmeleri (v1.4.5.9)

A. Deterministik Eşit İş (Equal-Work) Çatal Çözücü (`CBlockIndexWorkComparator`)

30 saniyelik işbirlikçi PoW/PoS hibrit zincirinde, ağ bölünmeleri veya eşzamanlı geri çekilme rotasyonları aynı yükseklikte ve eşit kümülatif zincir işine (`nChainWork`) sahip rakip bloklar üretebilir. Önceden `src/chain.h` içindeki `CBlockIndexWorkComparator` , blok özetlerini karşılaştırmadan önce yerel geliş sırasına (`nSequenceId`) bakıyordu; bu da bölünmüş alt ağlardaki düğümlerin bir sonraki blok kazılana kadar farklı zincir uçlarında kalmasına yol açabiliyordu.

- **Kural**: `pa->nChainWork == pb->nChainWork` olduğunda, karşılaştırıcı `nSequenceId` kontrolünden önce sayısal olarak daha küçük blok özetine (`pa->GetBlockHash() < pb->GetBlockHash()`) sahip blok indeksini deterministik olarak tercih eder.
- **Etki**: Ağ bölünmesi iyileştiğinde ve rakip bloklar yayıldığında, tüm düğümler bir sonraki bloğu beklemeden anında (`< 1s` içinde) aynı kanonik zincir ucunda birleşir.

B. LLMQ Çifte İmza Önleme (Anti-Equivocation) ve Çok Sekmeli (Multi-Hop) Gossip İletimi

- **Slot Bazlı Çifte İmza Koruması (`mapQuorumSignedSlot`)**: `src/llmq.cpp` içinde her LLMQ kurul üyesi `mapQuorumSignedSlot[std::make_pair(nHeight, rotationIndex)]` kaydını tutar. Bir kurul üyesi belirli bir `(nHeight, rotationIndex)` slotu için geçerli bir `PROPOSEBLOCK` imzaladığında, koordinatör rotasyon indeksi ilerlemediği sürece aynı slot için gelen rakip blok önerilerini imzalamayı reddeder. Bu sayede aynı slottaki iki rakip bloğun ikisinin birden 5'te 3 korum sertifikası (`vQuorumSig`) alması matematiksel olarak engellenir.
- **Çok Sekmeli Gossip İletimi (`mapSeenBlockProposals` , `mapSeenSigShares`)**: `PROPOSEBLOCK` ve `QUORUMSIGSHARE` P2P mesajları tekilleştirilerek tüm bağlı eşlere iletilir; böylece doğrudan (tek sekmede) bağlı olmayan korum üyeleri ve blok önericileri de öneri ve imza paylarını eksiksiz alır.
- **`ElectQuorumMembers` İçinde %100 Zincir Üstü (On-Chain) Yedek Havuz**: Son 2.880 blok içinde zincir üstü heartbeat ping göndermiş ve harcanmamış 4.200 KRISTA teminatına sahip Masternode sayısı `nQuorumSize` (5) değerinin altındaysa, `ElectQuorumMembers` fonksiyonu bellek içi P2P `mnodeman` durumuna başvurmadan doğrudan zincir üstü `GetAdamMinerPool(nHeight - 1, pView)` aday havuzuna deterministik olarak geri döner.

C. Doğrulanmış Zincir Üstü Masternode Heartbeat Ping Desteği (`ExtractOnChainMasternodePing` & `SendOnChainPing`)

- **Sıkıştırılmış (33-bayt) ve Sıkıştırılmamış (65-bayt) Açık Anahtar Desteği**: `src/adam.cpp` içindeki `ExtractOnChainMasternodePing` , serileştirilmiş `CDataStream` `OP_RETURN` kanıtıyla (`collateralOutpoint` + `Hash(collateralOutpoint || tx.vin[0].prevout)` üzerindeki `vchPingSig` imzası + isteğe bağlı `bool hasDelegation` + `COnChainMasternodeDelegation`) birlikte hem 36 baytlık (`0x21 <33-bayt sıkıştırılmış pubkey> OP_CHECKSIGVERIFY OP_1`) hem de 68 baytlık (`0x41 <65-bayt sıkıştırılmamış pubkey> OP_CHECKSIGVERIFY OP_1`) MESCAL `check-signature-verification` heartbeat betiklerini doğrular.

- **İletim Ücreti Muafiyeti (`0.0001 KRISTA`)**: Kimliği doğrulanmış zincir üstü Masternode ping işlemleri `GetMinRelayFee` (`src/main.cpp`) içindeki `1000x` `OP_RETURN` ücret çarpanından muaftır ve sabit `0.0001 KRISTA` işlem ücretiyle yayınlanır.

D. Kilitsiz (Lock-Free) Zincir Ucu Erişimi (`CChain::AtomicTip()`)

- `CChain` (`src/chain.h` / `src/chain.cpp`), `CChain::SetTip()` içinde güncellenen atomik bir `std::atomic<CBlockIndex*> pAtomicTip` işaretçisi tutar.
- `GetChainTip()` (`src/main.cpp`), `cs_main` kilidini almadan doğrudan `chainActive.AtomicTip()` değerini okur; böylece P2P mesaj işleme, RPC sorguları ve staking/madencilik iş parçacıkları arasındaki kilit çekişmesi (lock contention) ortadan kaldırılır.